

Výstup D6

Pracovný balík WP.2

Analýza implementácie štatistických metód do procesu kvantizácie

Vypracoval: Ing. Roman Budjač, PhD.

Táto správa sa zameriava na analýzu implementácie štatistických metód do procesu kvantizácie neurónových sietí. Koncept vychádza z rámca navrhnutého v štúdiu "Layer-wise Statistical Analysis: A Time-Sensitive Framework for Implementing Statistical Methods in Neural Network Training Sessions" (Budjač et.al. 2025, CSOC 2025), ktorá zavádza iný prístup k analýze a optimalizácii modelov v priebehu ich tréningu. Primárnym cieľom je využitie štatistických dát, získaných v reálnom čase, na fundamentálne informované rozhodovanie v procese kvantizácie.

Implementácia štatistických metód do kvantizačného procesu

Implementácia štatistických metód do procesu kvantizácie predstavuje paradigmatickú zmenu od tradičných jednotných kompresných prístupov k inteligentným, adaptívnym kvantizačným stratégiám, ktoré optimalizujú kompresiu pri zachovaní kvality modelu prostredníctvom rozhodnutí založených na dôkazoch. Táto transformácia vyžaduje fundamentálne prehodnotenie spôsobu, akým pristupujeme ku kompresii neurónových sietí, pričom namiesto aplikovania fixných kvantizačných schém implementujeme dynamický systém, ktorý sa adaptuje na jedinečné charakteristiky každého modelu a jeho jednotlivých vrstiev.

Charakterizovanie originálnych distribúcií váh začína systematickým štatistickým profilovaním pred akýmikoľvek kvantizačnými operáciami. Táto počiatočná fáza je kritická pre pochopenie inherentných vlastností modelu a stanovenie referenčných bodov pre následné monitorovanie degradácie. Proces využíva pokročilé štatistické analýzy implementované pomocou `scipy.stats` a `numpy` knižníc, ktoré poskytujú robustné nástroje pre numerické výpočty a štatistické modelovanie.

Prvým krokom v procese charakterizácie je extrakcia a predspracovanie distribúcií váh z jednotlivých vrstiev modelu. Tento proces je implementovaný v **Prílohe č. 1**, kde funkcia `extract_weight_distributions()` iteruje cez všetky vrstvy modelu a extrahuje váhové matice. Táto funkcia predstavuje základ celého analytického procesu, pretože kvalita extrahovaných dát priamo ovplyvňuje presnosť všetkých následných analýz.

Príloha č. 1: Extrakcia a predspracovanie váh

```
python
```

```
def extract_weight_distributions(model):  
    """Extrakcia a preprocessing distribúcií váh z vrstiev modelu"""  
    weight_distributions = {}  
  
    for i, layer in enumerate(model.layers):  
        if hasattr(layer, 'get_weights') and layer.get_weights():  
            weights = layer.get_weights()[0] # Získanie váhovej matice  
            flat_weights = weights.flatten() # Konverzia na 1D pole
```

```
# Odstránenie outlierov pomocou IQR metódy
q75, q25 = np.percentile(flat_weights, [75, 25])
iqr = q75 - q25
lower_bound = q25 - 1.5 * iqr
upper_bound = q75 + 1.5 * iqr
filtered_weights = flat_weights[(flat_weights >= lower_bound) &
                                (flat_weights <= upper_bound)]

weight_distributions[f'layer_{i}'] = {
    'raw_weights': flat_weights,
    'filtered_weights': filtered_weights,
    'shape': weights.shape
}

return weight_distributions
```

Ako vidíme v **Prílohe č. 1**, implementácia využíva metódu interkvartilového rozpätia (IQR) na identifikáciu a filtrovanie outlierov. Tento prístup je založený na robustných štatistických princípoch, kde hodnoty ležiace mimo 1,5-násobku IQR od prvého a tretieho kvartilu sú považované za potenciálne outlieri. Táto technika je obzvlášť vhodná pre prácu s distribúciami váh neurónových sietí, ktoré často vykazujú ťažké chvosty a môžu obsahovať extrémne hodnoty vzniknuté počas tréningu. Odstránenie týchto hodnôt je kritické pre zabezpečenie stability následných štatistických analýz, pričom súčasne zachováujeme väčšinu relevantných dát - typicky viac ako 99% hodnôt pri normálne distribuovaných dátach.

Po úspešnej extrakcii a filtrácii váh nasleduje komplexná štatistická charakterizácia prostredníctvom výpočtu distribučných momentov. Táto fáza poskytuje hlboký vhľad do štruktúry váhových distribúcií a je implementovaná v **Prílohe č. 2**. Funkcia `calculate_distribution_moments()` vypočítava štatistické metriky, ktoré charakterizujú vlastnosti váh v modeli.

Príloha č. 2: Výpočet štatistických momentov

```
python
def calculate_distribution_moments(weights):
    """Výpočet komplexných štatistických momentov"""
    moments = {
        'mean': np.mean(weights),
        'variance': np.var(weights),
        'std': np.std(weights),
        'skewness': stats.skew(weights),
```

```
'kurtosis': stats.kurtosis(weights),
'median': np.median(weights),
'mad': stats.median_abs_deviation(weights), # Median Absolute
Deviation
'range': np.ptp(weights), # Peak-to-peak range
'iqr': stats.iqr(weights), # Interquartile range
}

# Výpočet percentilov pre charakterizáciu distribúcie
percentiles = [1, 5, 10, 25, 50, 75, 90, 95, 99]
for p in percentiles:
    moments[f'p{p}'] = np.percentile(weights, p)

return moments
```

V **Prílohe č. 2** vidíme implementáciu výpočtu kľúčových štatistických momentov. Každá z týchto metrik poskytuje určitý pohľad na distribučné vlastnosti váh. Priemer a medián informujú o centrálnej tendencii, pričom ich vzájomný vzťah indikuje symetriu distribúcie. Rozptyl a štandardná odchýlka kvantifikujú variabilitu váh, čo je dôležité pre pochopenie, koľko informácii môže byť potenciálne stratených pri kvantizácii. Šikmosť (skewness) meria asymetriu distribúcie - pozitívne hodnoty naznačujú pravostranný chvost s možnými veľkými pozitívnymi váhami, zatiaľ čo negatívne hodnoty indikujú ľavostranný chvost. Špicatosť (kurtosis) poskytuje informácie o koncentrácii hodnôt okolo priemeru a hrúbke chvostov distribúcie. Vysoké hodnoty kurtosis často indikujú prítomnosť outlierov alebo distribúcie s ťažkými chvostami, čo môže významne komplikovať kvantizačný proces.

Mediánová absolútna odchýlka (MAD) predstavuje alternatívu k štandardnej odchýlke, ktorá je menej citlivá na extrémne hodnoty. Výpočet percentilov poskytuje detailný obraz o rozložení hodnôt naprieč celou distribúciou, čo umožňuje identifikovať koncentrácie váh v špecifických rozsahoch a identifikovať tak potenciálne problematické oblasti pre kvantizáciu.

Ďalšou kritickou súčasťou charakterizácie je analýza založená na entropii, ktorá poskytuje informačno-teoretický pohľad na distribúcie váh. Implementácia tejto analýzy je prezentovaná v **Prílohe č. 3**, kde funkcia `calculate_distribution_entropy()` vypočítava viaceré entropické miery.

Príloha č. 3: Výpočet entropických mier

```
python
```

```
def calculate_distribution_entropy(weights, bins='auto'):
    """Výpočet rôznych entropických mier"""
    # Generovanie histogramu pre výpočet entropie
```

```
hist, bin_edges = np.histogram(weights, bins=bins, density=True)
bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2

# Normalizácia histogramu na pravdepodobnostnú distribúciu
hist_normalized = hist / np.sum(hist)
hist_normalized = hist_normalized[hist_normalized > 0] # Odstránenie
nulových košov

entropy_measures = {
    'shannon_entropy': stats.entropy(hist_normalized, base=2),
    'differential_entropy': stats.differential_entropy(weights),
    'bin_count': len(hist),
    'effective_bins': np.sum(hist_normalized > 0.01), # Koše s >1%
    #pravdepodobnosťou
}

return entropy_measures, hist, bin_edges
```

Ako môžeme vidieť v **Prílohe č. 3**, výpočet entropie začína generovaním histogramu s automatickým určením počtu košov. Parameter `bins='auto'` využíva heuristiky implementované v NumPy, ktoré vyberajú optimálny počet košov na základe charakteristík dát. Normalizácia histogramu na pravdepodobnostnú distribúciu je kritická pre korektný výpočet Shannonovej entropie, ktorá kvantifikuje informačný obsah distribúcie v bitoch. Vyššie hodnoty entropie indikujú rovnomernejšie rozloženie váh naprieč celým rozsahom hodnôt, zatiaľ čo nižšie hodnoty naznačujú koncentráciu váh okolo špecifických hodnôt. Táto informácia je kľúčová pre predikciu vplyvu kvantizácie - distribúcie s nízkou entropiou môžu byť často kvantizované agresívnejšie bez významnej straty kvality.

Diferenciálna entropia rozširuje koncept entropie na spojité distribúcie a poskytuje komplementárny pohľad na informačný obsah. Metrika efektívnych košov, definovaná ako počet košov obsahujúcich viac ako 1% celkovej pravdepodobnosti, poskytuje praktickú mieru koncentrácie distribúcie, ktorá je intuitívnejšia než samotná entropia.

Generovanie referenčných histogramov predstavuje kľúčový medzník v procese charakterizácie, pretože vytvára štandardizovaný základ pre všetky následné porovnania. Implementácia v **Prílohe č. 4** demonštruje sofistikovaný prístup k vytvoreniu konzistentných histogramových reprezentácií naprieč všetkými vrstvami modelu.

Príloha č. 4: Generovanie referenčných histogramov

```
python
def generate_reference_histograms(weight_distributions, global_bins=100):
```

```
"""Generovanie štandardizovaných referenčných histogramov"""  
reference_data = {}  
  
# Výpočet globálneho rozsahu naprieč všetkými vrstvami  
all_weights = np.concatenate([data['raw_weights']  
                                for data in  
weight_distributions.values()])  
global_min, global_max = np.min(all_weights), np.max(all_weights)  
  
# Vytvorenie konzistentných hraníc košov pre všetky vrstvy  
bin_edges = np.linspace(global_min, global_max, global_bins + 1)  
  
for layer_name, data in weight_distributions.items():  
    weights = data['filtered_weights']  
  
    # Generovanie histogramu s konzistentným binningom  
    hist, _ = np.histogram(weights, bins=bin_edges, density=True)  
    hist_normalized = hist / np.sum(hist) # Normalizácia na  
pravdepodobnosť  
  
    reference_data[layer_name] = {  
        'histogram': hist_normalized,  
        'bin_edges': bin_edges,  
        'bin_centers': (bin_edges[:-1] + bin_edges[1:]) / 2,  
        'statistical_moments': calculate_distribution_moments(weights),  
        'entropy_measures': calculate_distribution_entropy(weights)[0]  
    }  
  
return reference_data
```

V **Prílohe č. 4** vidíme implementáciu, ktorá najprv vypočíta globálny rozsah hodnôt naprieč všetkými vrstvami pomocou spojenia všetkých váhových vektorov do jedného súvislého vektora. Tento prístup zabezpečuje, že všetky vrstvy budú mať histogramy s identickými hranicami intervalov, čo je nevyhnutné pre umožnenie priamych porovnaní medzi vrstvami. Použitie funkcie `np.linspace()` garantuje rovnomerné rozdelenie tried naprieč celým rozsahom hodnôt. Pre každú vrstvu sa následne generuje histogram s týmito globálnymi hranicami a normalizuje sa na pravdepodobnostnú distribúciu. Výsledná dátová štruktúra obsahuje nielen samotné histogramy, ale aj všetky podporné informácie potrebné pre následné analýzy, vrátane štatistických momentov a entropických mier vypočítaných pomocou funkcií z predchádzajúcich príloh.

Posledným krokom v základnej charakterizácii je stanovenie profilov citlivosti jednotlivých vrstiev na kvantizáciu. Tento proces, implementovaný v **Prílohe č. 5**, poskytuje kvantitatívne hodnotenie toho, ako rôzne vrstvy reagujú na rôzne úrovne kvantizácie.

Príloha č. 5: Stanovenie profilov citlivosti

python

```
def establish_sensitivity_profiles(model, reference_data,
weight_distributions,
                                test_bitwidths=[8, 6, 4]):
    """Stanovenie kvantizačnej citlivosti pre každú vrstvu"""
    sensitivity_profiles = {}

    for layer_name, ref_data in reference_data.items():
        layer_sensitivity = {}
        original_hist = ref_data['histogram']

        for bitwidth in test_bitwidths:
            # Aplikácia testovej kvantizácie
            original_weights =
weight_distributions[layer_name]['raw_weights']
            quantized_weights =
apply_uniform_quantization(original_weights, bitwidth)

            # Výpočet kvantizovaného histogramu
            quant_hist, _ = np.histogram(quantized_weights,
                                         bins=ref_data['bin_edges'],
                                         density=True)
            quant_hist_normalized = quant_hist / np.sum(quant_hist)

            # Výpočet citlivostných metrík
            kl_div = stats.entropy(original_hist + 1e-10,
quant_hist_normalized + 1e-10)
            js_div = calculate_js_divergence(original_hist,
quant_hist_normalized)

            layer_sensitivity[bitwidth] = {
                'kl_divergence': kl_div,
                'js_divergence': js_div,
                'sensitivity_score': (kl_div + js_div) / 2
```

}

```
sensitivity_profiles[layer_name] = layer_sensitivity
```

```
return sensitivity_profiles
```

Implementácia v **Prílohe č. 5** demonštruje systematický prístup k hodnoteniu citlivosti. Pre každú vrstvu a každú testovaciu šírku bitov (typicky 16,8 a 4 bitov) sa aplikuje uniformná kvantizácia a meria sa výsledná zmena distribúcie. Použitie identických hraníc košov z referenčných histogramov zabezpečuje konzistentné porovnanie. Výpočet KL divergencie pomocou `stats.entropy()` s pridaním malej epsilon hodnoty ($1e-10$) zabráňuje numerickým problémom pri práci s nulovými pravdepodobnosťami. JS divergencia poskytuje symetrickú mieru rozdielu medzi distribúciami, čo je užitočné pre holistické hodnotenie vplyvu kvantizácie. Kombinované skóre citlivosti, vypočítané ako priemer KL a JS divergencie, poskytuje jednotnú metriku pre porovnanie citlivosti rôznych vrstiev a uľahčuje rozhodovanie o optimálnych kvantizačných stratégiách.

Mechanizmus štatistickej spätnej väzby v reálnom čase

Mechanizmus štatistickej spätnej väzby v reálnom čase predstavuje jadro adaptívneho kvantizačného systému. Tento systém kontinuálneho monitorovania transformuje statický kvantizačný proces na dynamický, adaptívny systém schopný reagovať na zmeny v distribúciách váh počas kvantizácie. Implementácia využíva paralelnú architektúru, ktorá umožňuje simultánne sledovanie originálnych a kvantizovaných distribúcií. Základom tejto architektúry je trieda `ParallelDistributionTracker`, implementovaná v **Prílohe č. 6**, ktorá udržiava sofistikovaný dvojité buffer systém. Tento dizajn umožňuje nezávislé sledovanie oboch distribúcií pri zachovaní ich synchronizácie a konzistencie.

Príloha č. 6: Paralelné sledovanie distribúcií

python

```
class ParallelDistributionTracker:
    def __init__(self, reference_data):
        self.reference_data = reference_data
        self.current_distributions = {}
        self.quantized_distributions = {}

    def update_distributions(self, layer_name, original_weights,
                           quantized_weights):

        bin_edges = self.reference_data[layer_name]['bin_edges']

        # Generovanie histogramov s referenčným binningom
```



```
orig_hist, _ = np.histogram(original_weights, bins=bin_edges,
density=True)

quant_hist, _ = np.histogram(quantized_weights, bins=bin_edges,
density=True)

# Normalizácia na pravdepodobnostné distribúcie

self.current_distributions[layer_name] = orig_hist /
np.sum(orig_hist)

self.quantized_distributions[layer_name] = quant_hist /
np.sum(quant_hist)
```

Ako vidíme v **Prílohe č. 6**, trieda `ParallelDistributionTracker` inicializuje s referenčnými dátami získanými počas základnej charakterizácie. Táto architektúra zabezpečuje, že všetky následné porovnania používajú konzistentné metriky. Metóda `update_distributions()` predstavuje kľúčový mechanizmus pre udržiavanie aktuálnych distribúcií. Pri každej aktualizácii sa generujú nové histogramy pre obe distribúcie s použitím identických hraníc košov z referenčných dát. Tento prístup eliminuje potrebu dodatočných transformácií pri porovnávaní distribúcií a minimalizuje výpočtovú záťaž. Normalizácia histogramov na pravdepodobnostné distribúcie je kritická pre korektný výpočet divergenčných mier a zabezpečuje, že výsledky sú interpretovateľné a porovnateľné naprieč rôznymi vrstvami a časovými bodmi.

Výpočet štatistických metrík v reálnom čase je implementovaný v **Prílohe č. 7**, funkcia `calculate_real_time_divergences()`.

Príloha č. 7: Výpočet divergenčných mier v reálnom čase

python

```
def calculate_real_time_divergences(original_hist, quantized_hist,
epsilon=1e-10):

    """Výpočet divergenčných meraní s numerickou stabilitou"""

    # Pridanie epsilon pre numerickú stabilitu

    orig_stable = original_hist + epsilon
    quant_stable = quantized_hist + epsilon

    # Renormalizácia po pridaní epsilon

    orig_stable = orig_stable / np.sum(orig_stable)
    quant_stable = quant_stable / np.sum(quant_stable)

    divergences = {
        'kl_divergence': stats.entropy(orig_stable, quant_stable),
        'js_divergence': calculate_js_divergence_stable(orig_stable,
quant_stable),
```

```
'cosine_similarity': np.dot(orig_stable, quant_stable) /  
                        (np.linalg.norm(orig_stable) *  
np.linalg.norm(quant_stable)),  
    'wasserstein_distance': stats.wasserstein_distance(orig_stable,  
quant_stable)  
}  
  
return divergences  
  
def calculate_js_divergence_stable(p, q, epsilon=1e-10):  
    """Numericky stabilná Jensen-Shannon divergencia"""  
    # Výpočet strednej distribúcie  
    m = 0.5 * (p + q)  
  
    # Výpočet JS divergencie  
    js_div = 0.5 * (stats.entropy(p, m) + stats.entropy(q, m))  
    return js_div
```

V **Prílohe č. 7** vidíme implementáciu výpočtu divergenčných mier. Pridanie epsilon hodnoty ku všetkým prvkom histogramov pred výpočtom divergencií je esenciálne pre zabránenie numerickým nestabilitám, ktoré by vznikli pri logaritmovaní nulových hodnôt. Hodnota epsilon (1e-10) je zvolená dostatočne malá, aby neovplyvnila významne výsledky, ale dostatočne veľká na zabezpečenie numerickej stability aj pri práci s float32 presnosťou. Renormalizácia po pridaní epsilon zachováva vlastnosti pravdepodobnostnej distribúcie.

Funkcia vypočítava vybrané metriky: KL divergenciu pre asymetrické meranie informačnej straty, JS divergenciu pre symetrické porovnanie distribúcií, kosínusovú podobnosť pre uhlovú blízkosť distribučných vektorov, a Wassersteinovu vzdialenosť pre meranie "transportných nákladov" medzi distribúciami. Každá z týchto metrík poskytuje pohľad na zmeny v distribúciách a ich kombinácia umožňuje robustné hodnotenie vplyvu kvantizácie.

Záver:

Implementácia štatistických metód do procesu kvantizácie neurónových sietí predstavuje významný pokrok v oblasti optimalizácie modelov hlbokého učenia. Navrhnutý prístup na základe layer-wise štatistickej analýzy umožňuje prechod od tradičných uniformných kvantizačných techník k adaptívnym stratégiám, ktoré využívajú pokročilé štatistické analýzy pre optimalizáciu kompresie pri zachovaní kvality modelu. ezentovaná implementácia demonštruje, že integrácia pokročilých štatistických analýz priamo do kvantizačného procesu je technicky realizovateľná. Vytvorený framework poskytuje základ pre ďalší výskum v pracovných balíkoch WP.3 a WP.4.